

מחברת 1: ויקיפדיה כמקור נתונים ורשתות ידע

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

מה נלמד במחברת זו?

במחברת זו נלמד כיצד ויקיפדיה משמשת כמקור נתונים עבור מחקר מדעי הרוח, ולבנות רשת ידע אינטראקטיבית.

נושאים:

1. ויקיפדיה כמקור מידע – יתרונות ומגבלות
2. ה-API של מדיה-ויקי – גישה תכנותית
3. שליפת נתונים מקטגוריות עבריות
4. בניית רשת ידע עם NetworkX
5. ויזואליזציה אינטראקטיבית עם PyVis
6. ניתוח פערים – ערכים חסרים = פרויקט סיום!

 הקטגוריות שנחקר:

-  ארכיאולוגיה של ארץ ישראל
-  היסטוריה של עם ישראל
-  מדעי הרוח הדיגיטליים

מדריך שימוש במחברת

|||---|---|🕒 זמן מוערך | 🏠 בכיתה: ~50 דקות | 🏠 בבית: ~40 דקות נוספות || 🖥️ היכן להריץ |
Google Colab בלבד (כפתור ▶ למעלה) || 🌐 חיבור רשת | נדרש לחלק ב' ומעלה (שליפת ויקיפדיה) |

סדר ריצה:

חשוב: הריצו תאים מלמעלה למטה, בסדר. אל תדלגו!

- תא 1 → התקנת ספריות (✓) (חכו עד שמופיע ✓)
תא 2 → יבוא ספריות (✓) (חכו עד שמופיע ✓)
(ערכים X חכו עד שמופיע ✓ (ונמצאו) תא 3 → הגדרת המחלקה
וכן הלאה...

קיבלתם שגיאה? 

1. קראו את הודעת השגיאה – בדרכ כלל מסבירה מה קרה
2. הריצו מחדש את התא (לחצו ► שוב)
3. שאלו את Gemini: העתיקו את הודעת השגיאה לשורת החיפוש של Gemini ורשמו "מה אומרת שגיאה זו בפיתון ואיך לפתור?"
4. שאלו את המרצה אם הבעיה נמשכת

חלק א: ויקיפדיה כמקור נתונים מחקרי

מדוע ויקיפדיה מעניינת את מדעי הרוח הדיגיטליים?

ויקיפדיה אינה רק אנציקלופדיה – היא מסד נתונים ענק של ידע אנושי מובנה:

מאפיין	פרטים
היקף	6.7M~ ערכים באנגלית, 380K~ בעברית
קישוריות	כל ערך מקושר לאחרים = רשת ידע
קטגוריות	ארגון היררכי של ידע
היסטוריית עריכות	ניתן לחקור התפתחות ידע לאורך זמן
API חינומי	גישה פרוגרמטית מלאה

שאלות מחקריות:

- אילו נושאים מיוצגים-חסר (underrepresented)?
- כיצד מתפתח הידע לאורך זמן?
- מהי הרשת של קשרים בין מושגים?
- מה ה"ליבה" של שדה ידע?

📖 לקריאה: Niederer & van Dijck (2010). *Wisdom of the crowd or technicity of*

content? New Media & Society

```
In [ ... # == התקנת ספריות == 5 שקף ==
# התקנת ספריות נדרשות
# % pip = מתקין לתוך הסביבה הנוכחית
%pip install pyvis wikipedia-api tqdm python-bidi scipy -q

print("✅ הספריות הותקנו")
print("• pyvis - ויזואליזציה אינטראקטיבית")
print("• wikipedia-api - גישה ל-API")
print("• tqdm - סרגל התקדמות")
print("• python-bidi - תצוגת עברית תקינה בגרפים")
print("• scipy - ומידות מרכזיות PageRank חישובי")
```

```

1.0.62 <- 1.1.52 :elbaliava si pip fo esaeler wen A [eciton]
i pip m- 01.3nohtyp/nib/01.3@nohtyp/tpo/werbemoh/tpo/ :nur ,etadpu oT [eciton]
pip edargpu-- llatsn
.segakcap detadpu esu ot lenrek eht tratser ot deen yam uoy :etoN

```

- sivyp - ויזואליזציה אינטראקטיבית
- ipa-aidepikiw - גישה ל-IPA
- mdqt - סרגל התקדמות
- idib-nohtyp - תצוגת עברית תקינה בגרפים
- ypics - חיסובי knaRegaP ומידות מרכזיות

```

In [ ... # == יבוא ספריות == 6 שקף ==
# יבוא ספריות
import requests # שליפת נתונים מהרשת
import json # עיבוד JSON
import pandas as pd # ניהול נתונים בטבלאות
import networkx as nx # בניית ועיבוד גרפים
from pyvis.network import Network # ויזואליזציה אינטראקטיבית
import time # עיכובים בין בקשות
from tqdm.auto import tqdm # סרגל התקדמות (גם ללא ipywidgets)
from IPython.display import HTML, display
import matplotlib.pyplot as plt
import matplotlib
import os
from bidi.algorithm import get_display # תיקון כיוון טקסט עברי בגרפים

matplotlib.rcParams['axes.unicode_minus'] = False

print("✓ כל הספריות יובאו!")
print(f" NetworkX {nx.__version__} | Pandas {pd.__version__}")

```

✓ כל הספריות יובאו!

3.3.2 sadnaP | 2.4.3 XkrowteN

```

gorPI :gninraWmdqT :12:yp.otua/mdqt/segakcap-etis/01.3nohtyp/bil/werbemoh/tpo/
r.stegdiwypi//:sptth eeS .stegdiwypi dna retypuj etadpu esaelP .dnuof ton sser
lmth.llatsni_resu/elbats/ne/oi.scodehtdae
mdqt_koobeton sa mdqt tropmi koobetonotua. morf

```

חלק ב: ה-API של ויקיפדיה

מהו API?

API (Application Programming Interface) = ממשק תכנות יישומים.
חשבו עליו כ"תפריט" – אנחנו בוחרים מה לבקש, ה-API מחזיר את הנתונים.

כתובת הבסיס של ויקיפדיה העברית:

<https://he.wikipedia.org/w/api.php>

דוגמה לבקשה:

https://he.wikipedia.org/w/api.php?
action=query&list=categorymembers&cmttitle=של_קטגוריה:ארכאולוגיה_של_קטגוריה:
&format=json ארץ_ישראל

פרמטרים עיקריים:

תיאור	פרמטר
שאלת מידע	action=query
רשימת חברי קטגוריה	list=categorymembers
קישורים בתוך ערך	prop=links
תוכן/תקציר הערך	prop=extracts
פורמט התגובה	format=json

 https://www.mediawiki.org/wiki/API:Main_page תיעוד מלא:

כלים מקוונים ללא קוד – לסיור ראשון בויקיפדיה

לפני שנכתוב קוד, כדאי לדעת שאפשר לבצע חלק מהשאלות שנלמד היום ישירות בדפדפן – ללא שורת Python אחת.

כלים מומלצים

כלי	מה אפשר לעשות	קישור
Six Degrees of Wikipedia	מוצא נתיב קצר בין שני ערכים – ויזואלי ומרשים!  אנגלית בלבד	Megiddo → Babylonian captivity
PetScan	שאלות מתקדמות על קטגוריות – כבר מוגדר לוויקיפדיה העברית	ארכאולוגיה של ארץ ישראל בפטסקן
API Sandbox (ויקיפדיה)	ניסוי בקשות API ישירות ב-Wikipedia ללא קוד	מיזח:ApiSandbox
דף הקטגוריה עצמו	ריצה ידנית – רשימת כל הערכים בקטגוריה	ארכאולוגיה של ארץ ישראל

הערה על **Six Degrees**: הכלי עובד רק על ויקיפדיה האנגלית. כדי לחפש בעברית השתמשו ב-PetScan או בדף הקטגוריה.

דוגמה להדגמה בכיתה – Six Degrees

פתחו את הקישור: [Megiddo → Babylonian captivity](#) (באנגלית)

תוכלו לראות בדיוק אחד הedges שהגרף שלנו יבנה – בצורה ויזואלית ואינטואיטיבית.

שאלה לדין: כמה "צעדים" לוקח להגיע מ-Megiddo ל-Babylonian captivity?
נוס גם: Social network analysis → Megiddo . מה מגלה המסלול על המבנה
של ידע בוויקיפדיה?

In [...

```
# == בדיוקת חיבור HebrewWikipedia + שקפים 8-7 == מחלקת ==  
# =====  
# מחלקת גישה לוויקיפדיה העברית  
# =====  
  
class HebrewWikipedia:  
    """  
    MediaWiki API. ממשיך לוויקיפדיה העברית דרך  
  
    שימוש:  
    wiki = HebrewWikipedia()  
    pages = wiki.get_category_members("ארכלולוגיה של ארץ ישראל")  
    links = wiki.get_page_links("מגידו")  
    """  
  
    BASE_URL = "https://he.wikipedia.org/w/api.php"  
  
    def __init__(self, delay=0.15):  
        """  
        אתחול הממשק.  
        delay: עיכוב (שניות) בין בקשות - כיבוד מגבלות השרת  
        """  
        self.session = requests.Session()  
        self.delay = delay  
        # חשוב: זיהוי הבוט - כיבוד נוהלי ויקיפדיה  
        self.session.headers.update({  
            'User-Agent': 'IntroDH-Course-Bot/1.0 (Ariel University; Educational  
            'Contact: shygordin@gmail.com'  
        })  
  
    def _request(self, params):  
        """עם טיפול בשגיאות. פונקציה פנימית API-שליחת בקשה ל"""  
        params['format'] = 'json'  
        # API-ויקיפדיה מצפה לקווים תחתונים (לא רווחים) בשמות הדפים בפרמטרי ה  
        for key in ('cmtitle', 'titles', 'page'):  
            if key in params:  
                params[key] = params[key].replace(' ', '_')  
        try:  
            resp = self.session.get(self.BASE_URL, params=params, timeout=30)  
            resp.raise_for_status()  
            time.sleep(self.delay)  
            return resp.json()  
        except Exception as e:  
            print(f"⚠ שגיאה: {e}")  
            return {}  
  
    def get_category_members(self, category_name, depth=1, max_pages=None):  
        """  
        שליפת כל הערכים בקטגוריה (כולל תת-קטגוריות)
```

פרמטרים:

```
category_name : שם הקטגוריה ללא הקידומת "קטגוריה"  
depth         : עומק חיפוש בתת-קטגוריות (ברירת מחדל: 1)  
               קטגוריות גדולות כמו ארכיאולוגיה דורשות  
max_pages     : (ללא מגבלה = None) מגבלת ערכים
```

dict מחזיר: title, pageid, category
"""

```
pages = []
```

```
seen_cats = set()
```

```
def _fetch(cat, current_depth):
```

```
    if cat in seen_cats or current_depth > depth:
```

```
        return
```

```
    if max_pages and len(pages) >= max_pages:
```

```
        return
```

```
    seen_cats.add(cat)
```

```
    continue_key = None
```

```
    while True:
```

```
        params = {
```

```
            'action': 'query',
```

```
            'list': 'categorymembers',
```

```
            'cmtitle': f'קטגוריה:{cat}',
```

```
            'cmlimit': 500,
```

```
            'cmtype': 'page|subcat'
```

```
        }
```

```
        if continue_key:
```

```
            params['cmcontinue'] = continue_key
```

```
        data = self._request(params)
```

```
        if 'query' not in data:
```

```
            break
```

```
        for item in data['query']['categorymembers']:
```

```
            if item['ns'] == 0: # ערך רגיל
```

```
                if not any(p['title'] == item['title'] for p in pages):
```

```
                    pages.append({
```

```
                        'title': item['title'],
```

```
                        'pageid': item['pageid'],
```

```
                        'category': category_name
```

```
                    })
```

```
                elif item['ns'] == 14 and current_depth < depth: # קטגוריה
```

```
                    subcat = item['title'].replace('קטגוריה:', '')
```

```
                    _fetch(subcat, current_depth + 1)
```

```
        if 'continue' in data:
```

```
            continue_key = data['continue']['cmcontinue']
```

```
        else:
```

```
            break
```

```
    _fetch(category_name, 0)
```

```
    return pages
```

```
def get_page_links(self, title, filter_titles=None):
```

```
    """
```

שליפת קישורים פנימיים מתוך ערך.

פרמטרים:

title : כותרת הערך
filter_titles : אם סופק, מחזיר רק קישורים לכותרות אלו

מחזיר: רשימת כותרות מקושרות
"""

links = []

continue_key = None

while True:

params = {
 'action': 'query',
 'prop': 'links',
 'titles': title,
 'pllimit': 500,
 'plnamespace': 0
}

if continue_key:
 params['plcontinue'] = continue_key

data = self._request(params)

if 'query' not in data:
 break

for pid, pdata in data['query']['pages'].items():
 for link in pdata.get('links', []):
 t = link['title']
 if filter_titles is None or t in filter_titles:
 links.append(t)

if 'continue' in data:
 continue_key = data['continue']['plcontinue']
else:
 break

return links

def get_page_summary(self, title):

"""שליפת תקציר ומידע בסיסי על ערך."""

data = self._request({
 'action': 'query',
 'prop': 'extracts|info',
 'titles': title,
 'exintro': True,
 'exchars': 800,
 'inprop': 'url'
})

if 'query' not in data:
 return {}

for pid, pdata in data['query']['pages'].items():
 if pid != '-1':
 return pdata

return {}

```

def check_exists(self, titles):
    """
    בדיקת קיום רשימת ערכים.
    dict: {כותרת: True/False}
    50 עובד באצוות של
    """
    results = {}
    for i in range(0, len(titles), 50):
        batch = titles[i:i+50]
        data = self._request({
            'action': 'query',
            'prop': 'info',
            'titles': '|'.join(batch)
        })
        if 'query' not in data:
            continue
        for pid, pdata in data['query']['pages'].items():
            exists = pid != '-1' and 'missing' not in pdata
            results[pdata['title']] = exists
    return results

def get_red_links(self, title):
    """
    שליפת קישורים אדומים (ערכים מוזכרים אך לא קיימים).
    אלו הם פערי הידע בוויקיפדיה
    """
    data = self._request({
        'action': 'parse',
        'page': title,
        'prop': 'links'
    })
    missing = []
    if 'parse' in data:
        for link in data['parse'].get('links', []):
            if link.get('ns') == 0 and 'exists' not in link:
                m = link.get('*', '')
                if m:
                    missing.append(m)
    return missing

# — יצירת אוֹבֵּיֶקְט וִיקִיפֶדִיָּה —
wiki = HebrewWikipedia(delay=0.15)
print("✅ ממשיך וִיקִיפֶדִיָּה מוכן!")
print()
print("🔍 בודק חיבור לוויקיפדיה")
print("(שולף כמה ערכים לבדיקה – עשוי לקחת 10–20 שניות)")
try:
    test = wiki.get_category_members("ארכאולוגיה של ארץ ישראל", depth=1, max_page_size=10)
    print(f"✅ (מבדיקת מדגם) {len(test)} החיבור תקין! נמצאו")
    if test:
        print(f"דוגמה: {test[0]['title']}, {test[1]['title']} if len(test)>1")
    if len(test) == 0:
        print("⚠️ 0 נסו. DEMO_MODE=True ערכים – ייתכן שהאינטרנט חסום.")
except Exception as exc:
    print(f"⚠️ בעיית רשת: {exc}")

```



```
print(" → בדקו את חיבור האינטרנט ונסו שוב ")
print(" → בתא הבא DEMO_MODE-אם הבעיה נמשכת, השתמשו ב "
```

ממשק ויקיפדיה מוכן! ✓

בודק חיבור לוויקיפדיה... 🔍

(שולף כמה ערכים לבדיקה - עשוי לקחת 01-02 שניות)

✓ החיבור תקין! נמצאו 75 ערכים (מבדיקת מדגם)

דוגמה: ארכאולוגיה של ארץ ישראל, eicinéhP ed noissiM

In [...

```
# == שקף 9 == הגדרת קטגוריות ==
# =====
# הגדרת הקטגוריות ונתוני הויזואליזציה
# =====

CATEGORIES = {
    'ארכאולוגיה של ארץ ישראל': {
        'color': '#e74c3c',
        'bg_color': '#fadbd8',
        'icon': '🏛️',
        'group': 1,
        'depth': 1,          # קטגוריה שטוחה - מאמרים ישירים ברמה 1
        'max_pages': 100
    },
    'תולדות עם ישראל': {
        'color': '#2980b9',
        'bg_color': '#d6eaf8',
        'icon': '🏡',
        'group': 2,
        'depth': 1,          # קטגוריה קיימת ועם 50~ ערכים ברמה ראשונה
        'max_pages': 100
    },
    'מדעי הרוח הדיגיטליים': {
        'color': '#27ae60',
        'bg_color': '#d5f5e3',
        'icon': '🖥️',
        'group': 3,
        'depth': 1,          # קטגוריה שטוחה - מאמרים ישירים
        'max_pages': 80
    }
}

print("🏠 קטגוריות מוגדרות:")
for cat, info in CATEGORIES.items():
    print(f" {info['icon']} {cat} (עומק={info['depth']})")
```

קטגוריות מוגדרות: 🏠

ארכאולוגיה של ארץ ישראל (עומק=1) 🏛️

תולדות עם ישראל (עומק=1) 🏡

מדעי הרוח הדיגיטליים (עומק=1) 🖥️

חלק ב' - שליפת נתונים (בכיתה יחד) 🏠

🕒 הערה לסבלנות: השליפה הבאה שולחת מאות בקשות לשרת ויקיפדיה ועשויה לקחת 5-10

דקות.

זה נורמלי! ראו את הסרגל מתקדם ואל תסגרו את הדפדפן.

💡 אם אתם לא בחיבור אינטרנט טוב, הדליקו את **DEMO_MODE=True** בתא הבא — זה ייתן לכם נתוני דוגמה להמשך.

```
In [ ... # == מצב עבודה == 10 שקף (DEMO_MODE) ==  
# =====  
# הגדרת מצב עבודה  
# =====  
# אם True-שנו ל:  
#   - אין חיבור אינטרנט תקין  
#   - השליפה לוקחת יותר מדי זמן  
#   - רוצים לתרגל ניתוח על נתונים מוכנים  
  
DEMO_MODE = False # כדי לדלג על שליפת ויקיפדיה True-שנו ל ←  
  
# DEMO_MODE=True (ישמשו כאשר) נתוני דוגמה  
DEMO_PAGES = [  
    {'title': 'ארכאולוגיה של ארץ ישראל', 'pageid': 51200, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'תל חצור', 'pageid': 57301, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'תל לכיש', 'pageid': 42100, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'בית שאן', 'pageid': 38200, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'תל באר שבע', 'pageid': 33100, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'יריחו', 'pageid': 29400, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'אשדוד', 'pageid': 26300, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'תקופת הברזל', 'pageid': 22100, 'category': 'ארכאולוגיה של ארץ ישראל'},  
    {'title': 'גלות בבל', 'pageid': 71200, 'category': 'תולדות עם ישראל'},  
    {'title': 'ממלכת ישראל', 'pageid': 65400, 'category': 'תולדות עם ישראל'},  
    {'title': 'ממלכת יהודה', 'pageid': 62300, 'category': 'תולדות עם ישראל'},  
    {'title': 'שיבת ציון', 'pageid': 58900, 'category': 'תולדות עם ישראל'},  
    {'title': 'המרד הגדול', 'pageid': 54100, 'category': 'תולדות עם ישראל'},  
    {'title': 'ספרות עברית', 'pageid': 49200, 'category': 'תולדות עם ישראל'},  
    {'title': 'דיגיטליזציה', 'pageid': 31000, 'category': 'מדעי הרוח הדיגיטליים'},  
    {'title': 'קריאה מרחוק', 'pageid': 28500, 'category': 'מדעי הרוח הדיגיטליים'},  
    {'title': 'ניתוח רשתות חברתיות', 'pageid': 25100, 'category': 'מדעי הרוח הדיגיטליים'}  
]  
  
# נתוני קשרים לדוגמה (אמולציה של קישורים בין ערכים)  
DEMO_EDGES = [  
    ('תל חצור', 'תל לכיש'), ('תקופת הברזל', 'תקופת הברזל'), ('מגידו', 'תל חצור'),  
    ('ממלכת יהודה', 'ממלכת ישראל'), ('גלות בבל', 'ממלכת יהודה'), ('ממלכת ישראל', 'ממלכת יהודה'),  
    ('קריאה מרחוק', 'קריאה מרחוק'), ('דיגיטליזציה', 'ממלכת יהודה'), ('המרד הגדול', 'ממלכת יהודה'),  
    ('תל באר שבע', 'מגידו'), ('בית שאן', 'קריאה מרחוק'), ('ניתוח רשתות חברתיות', 'קריאה מרחוק')  
]  
  
if DEMO_MODE:  
    print("🟡 מצב דמו פעיל - משתמשים בנתוני דוגמה")  
else:  
    print("🟢 מצב חי - נשלף נתונים אמיתיים מוויקיפדיה")
```

🟢 מצב חי - נשלף נתונים אמיתיים מוויקיפדיה

In [...

```
# == שקף 11 == שליפת נתונים מוויקיפדיה ==
# =====
# שליפת כל הערכים מוויקיפדיה (או טעינת נתוני דמו)
# =====
# 🕒 במצב חי: תהליך זה עשוי לקחת 5-10 דקות
# !ראו את הסרגלים מתקדמים - אל תפסיקו את הריצה

import time as _time
_start = _time.time()

if DEMO_MODE:
    # ----- מצב דמו -----
    all_pages = DEMO_PAGES.copy()
    df = pd.DataFrame(all_pages)
    print("🟡 נטענו נתוני דמו:")
    for cat in df['category'].unique():
        n = len(df[df['category']==cat])
        print(f"    {CATEGORIES[cat]['icon']} {cat}: {n} ערכים")
    print(f"📊\nסה"כ: {len(df)} (דמו) ערכים")
    display(df.head(8))

else:
    # ----- מצב חי -----
    print("🚀...שולף ערכים מוויקיפדיה העברית")
    print("==" * 55)
    print("⌚!השליפה עשויה לקחת 5-10 דקות - המתינו בסבלנות")
    print()

    all_pages = []
    category_pages = {}

    for cat_name, cat_info in CATEGORIES.items():
        cat_depth = cat_info.get('depth', 1)
        cat_max = cat_info.get('max_pages', None)
        print(f"\n{cat_info['icon']} שולף: {cat_name}")
        print(f"    (ערכים={'ללא הגבלה' if cat_max is None else cat_max} מקסימום, עומק={cat_depth})")

        pages = wiki.get_category_members(cat_name, depth=cat_depth, max_pages=
category_pages[cat_name] = pages
        all_pages.extend(pages)

        print(f"    ✓ נמצאו {len(pages)} ערכים")

    print("\n" + "==" * 55)
    print(f"📊\nסה"כ: {len(all_pages)} ערכים")

    # יצירת DataFrame
    df = pd.DataFrame(all_pages).drop_duplicates('title').reset_index(drop=True)
    print(f"    ערכים ייחודיים: {len(df)} :לאחר הסרת כפילויות")
    print()
    display(df.head(8))

elapsed = _time.time() - _start
print(f"🕒\nזמן שחלף: {elapsed:.0f} שניות")
```

```
print("🟢! נתוני הבסיס מוכנים - ממשיכים לניתוח")
```

שולף ערכים מוויקיפדיה העברית...

השליפה עשויה לקחת 5-01 דקות - המתינו בסבלנות!

שולף: ארכאולוגיה של ארץ ישראל
(עומק=1, מקסימום=001 ערכים)
✓ נמצאו 401 ערכים

שולף: תולדות עם ישראל
(עומק=1, מקסימום=001 ערכים)
✓ נמצאו 541 ערכים

שולף: מדעי הרוח הדיגיטליים
(עומק=1, מקסימום=08 ערכים)
✓ נמצאו 95 ערכים

סה"כ: 803 ערכים
לאחר הסרת כפילויות: 803 ערכים ייחודיים

	category	pageid	title
0	ארכאולוגיה של ארץ ישראל	368147	ארכאולוגיה של ארץ ישראל
1	ארכאולוגיה של ארץ ישראל	2153878	Mission de Phénicie
2	ארכאולוגיה של ארץ ישראל	2176801	אבן מגדלא
3	ארכאולוגיה של ארץ ישראל	774961	אמות המים של קיסריה
4	ארכאולוגיה של ארץ ישראל	18139	ארכאולוגיה מקראית
5	ארכאולוגיה של ארץ ישראל	964619	ארכאומטלורגיה אל-ברזלית בדרום הלבנט
6	ארכאולוגיה של ארץ ישראל	475661	אתרי המקרא
7	ארכאולוגיה של ארץ ישראל	677100	בביתא

זמן שולף: 3 שניות
נתוני הבסיס מוכנים - ממשיכים לניתוח! 🟢

```
In [ ... # ===== שקף 12 ===== גרף התפלגות קטגוריות =====  
# =====  
# ניתוח ראשוני: גרף התפלגות  
# =====  
  
counts = df.groupby('category').size().sort_values(ascending=False)  
  
print("🇮🇱 התפלגות לפי קטגוריה")  
for cat, n in counts.items():  
    icon = CATEGORIES[cat]['icon']  
    bar_str = '■' * (n // 5)  
    print(f" {icon} {cat[:35]:<35} {n:>4} |{bar_str}")
```

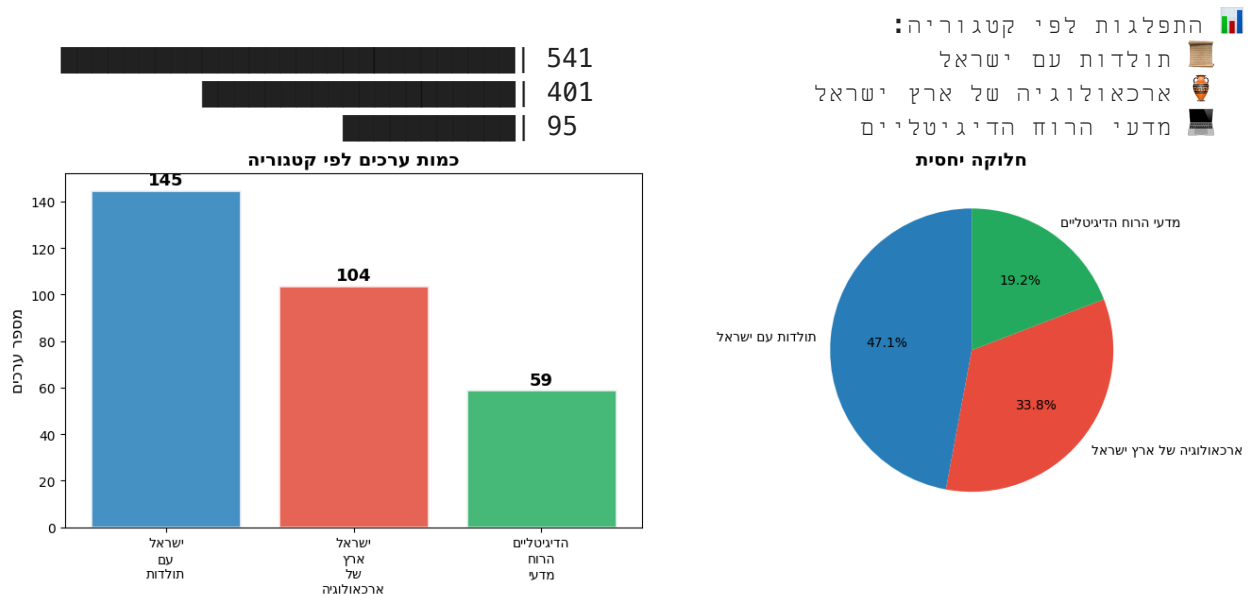
```
# גרף
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
colors = [CATEGORIES[c]['color'] for c in counts.index]

# python-bidi באמצעות matplotlib-תיקון כיוון עברית ב
def rtl(text):
    """matplotlib מכין מחזורות עברית לתצוגה תקינה בגרף"""
    return get_display(text)

# עמודות
bars = ax1.bar(range(len(counts)), counts.values,
               color=colors, alpha=0.85, edgecolor='white', linewidth=2)
ax1.set_xticks(range(len(counts)))
ax1.set_xticklabels([rtl(c).replace(' ', '\n') for c in counts.index], fontsize=12)
ax1.set_ylabel(rtl('מספר ערכים'), fontsize=12)
ax1.set_title(rtl('כמות ערכים לפי קטגוריה'), fontsize=14, fontweight='bold')
for bar, val in zip(bars, counts.values):
    ax1.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.5,
             str(val), ha='center', va='bottom', fontsize=13, fontweight='bold')

# עוגה
ax2.pie(counts.values, labels=[rtl(c) for c in counts.index], colors=colors,
        autopct='%1.1f%%', startangle=90,
        textprops={'fontsize': 10})
ax2.set_title(rtl('חלוקה יחסית'), fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('01_category_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print("📁 01 הגרף נשמר: category_distribution.png")
```



הגרף נשמר: gnp.noitubirtsid_yrogetac_10

חלק ג: תיאוריית רשתות – Network Analysis

מה זה גרף / רשת?

גרף הוא מבנה מתמטי המורכב מ:

- **צמתים** (Nodes): הישויות – כאן, ערכי ויקיפדיה
- **קשתות** (Edges): הקשרים – קישורים בין ערכים

מדדי מרכזיות:

שימוש	הסבר	מדד
"כמה פופולרי"	כמה ערכים מקשרים לכאן	In-Degree
"כמה מחובר"	כמה ערכים הערך מקשר אליהם	Out-Degree
"כמה סמכותי"	אלגוריתם גוגל – חשיבות יחסית	PageRank
"גשר בין קבוצות"	כמה פעמים על נתיב קצר	Betweenness

גרף מכוון לעומת לא מכוון:

- **מכוון (Directed)**: קישור מ-א ל-ב $\neq$ קישור מ-ב ל-א
- **לא מכוון**: כל קישור הוא דו-כיווני

ויקיפדיה = גרף מכוון (ערך א' מקשר לב', אבל לא בהכרח ב' מקשר לא').

מומלץ: Barabási, A.L. (2016). *Network Science*. (חינם: networksciencebook.com)

In [...

```
# == שקף 14 == בניית רשת הידע ==
# =====
# בניית רשת הידע
# =====
# 📖 בכיתה: הריצו תא זה ועקבו אחר הסרגלים
# 🕒 במצב חי: עד 10 דקות. במצב דמו: מהיר מאוד

print("🕸️ בונה רשת ידע")
print("=" * 55)

G = nx.DiGraph()

all_titles = set(df['title'].tolist())

# — שלב 1: הוספת צמתים —
print("🔪 מוסיף צמתים...")
for _, row in df.iterrows():
    cat = row['category']
    G.add_node(
        row['title'],
        category=cat,
        color=CATEGORIES[cat]['color'],
        group=CATEGORIES[cat]['group'],
        pageid=str(row['pageid'])
    )
print(f"👤 {G.number_of_nodes()} צמתים נוספו")
```

```
# — שלב 2: שליפת/הגדרת קשתות —
if DEMO_MODE:
    print(f"\n🔗 מוסיף קשתות מנתוני דמו...")
    for src, tgt in DEMO_EDGES:
        if src in all_titles and tgt in all_titles:
            G.add_edge(src, tgt)
    print(f"📊 {G.number_of_edges()} קשתות נוספו")
else:
    print(f"\n🔗 ערכים {len(all_titles)} שולף קישורים עבור...")
    print(f"📊 (שולף רק קישורים בין ערכים ברשימה שלנו)")
    print(f"⌚ עשוי לקחת כמה דקות נוספות...")
    with tqdm(total=len(all_titles), desc="שליפת קישורים") as pbar:
        for title in list(all_titles):
            links = wiki.get_page_links(title, filter_titles=all_titles)
            for link in links:
                if link != title:
                    G.add_edge(title, link)
            pbar.update(1)

print(f"\n✅ הרשת נבנתה!")
print(f"📊 צמתים : {G.number_of_nodes():,}")
print(f"📊 קשתות : {G.number_of_edges():,}")
print(f"📊 צפיפות: {nx.density(G):.4f}")
print()
print(f"💡 מה אומרת הצפיפות?")
print(f"📊 צפיפות {nx.density(G):.4f} = {nx.density(G)*100:.1f}% קיימים")
print(f"📊 רשת ריאלית תמיד דלילה - ערכים לא מקושרים לכולם")
```

בונה רשת ידע... 🧠

=====

מוסיף צמתים... 📌

803 צמתים נוספו

🔗 שולף קישורים עבור 803 ערכים...
 (שולף רק קישורים בין ערכים ברשימה שלנו)
 ⌚ עשוי לקחת כמה דקות נוספות...

[s/ti05.2 ,00:00>30:20] 803/803 | ██████████ | %001 שליפת קישורים:

✅ הרשת נבנתה!

צמתים : 803

קשתות : 217,1

צפיפות: 1810.0

💡 מה אומרת הצפיפות?

צפיפות 1810.0 = 8.1% מהקשרים האפשריים קיימים

רשת ריאלית תמיד דלילה - ערכים לא מקושרים לכולם!

In [...

```
# == 15 שקף == PageRank מרכזיות ==
# =====
# ניתוח סטטיסטי - מדדי מרכזיות
# =====

print(f"📊 מחשב מדדי מרכזיות...")

in_deg = dict(G.in_degree())
out_deg = dict(G.out_degree())
pagerank = nx.pagerank(G, alpha=0.85)
```

```

# הוספה כתכונות צמתים
for n in G.nodes():
    G.nodes[n]['in_degree'] = in_deg.get(n, 0)
    G.nodes[n]['out_degree'] = out_deg.get(n, 0)
    G.nodes[n]['degree'] = in_deg.get(n, 0) + out_deg.get(n, 0)
    G.nodes[n]['pagerank'] = round(pagerank.get(n, 0), 6)

# טבלת נתונים
stats = []
for n in G.nodes():
    nd = G.nodes[n]
    stats.append({
        'כותרת': n,
        'קטגוריה': nd.get('category', ''),
        'קישורים_נכנסים': nd['in_degree'],
        'קישורים_יוצאים': nd['out_degree'],
        'סך_קישורים': nd['degree'],
        'PageRank': nd['pagerank']
    })

df_stats = pd.DataFrame(stats).sort_values('PageRank', ascending=False).reset_index()
df_stats.to_csv('02_network_stats.csv', index=False, encoding='utf-8-sig')

print("\n★ 15 הערכים המרכזיים ביותר (PageRank):")
print("-" * 70)
for i, row in df_stats.head(15).iterrows():
    icon = CATEGORIES.get(row['קטגוריה'], {}).get('icon', '•')
    print(f" {i+1:2}. {icon} {row['כותרת'][:42]:<42} PR={row['PageRank']:.5f}")

isolated = [n for n in G.nodes() if G.degree(n) == 0]
print(f"\n🔴 (אין קישורים לאחרים) ערכים מבודדים: {len(isolated)}")

print("\n📄 02 סטטיסטיקות נשמרו: network_stats.csv")

```


51 הערכים המרכזיים ביותר (knaRegaP) :

24920.0=RP	פרויקט גוטנברג	.1
09620.0=RP	ארכיון האינטרנט	.2
71620.0=RP	פרויקט בן-יהודה	.3
50420.0=RP	דעת (אתר אינטרנט)	.4
28320.0=RP	ויקיטקסט	.5
70320.0=RP	אנטישמיות	.6
58120.0=RP	היברובוקס	.7
33020.0=RP	ספרייה דיגיטלית	.8
86910.0=RP	גוגל ספרים	.9
34810.0=RP	פרויקט השו"ת	.01
04810.0=RP	אוצר הספרים היהודי השיתופי	.11
58710.0=RP	ספריית אסיף	.21
07710.0=RP	המילון ההיסטורי ללשון העברית	.31
56710.0=RP	אוצר החכמה	.41
53710.0=RP	על התורה	.51

ערכים מבודדים (אין קישורים לאחריהם): 04

סטטיסטיקות נשמרו: vsc.stats_krowten_20

חלק ד: ויזואליזציה אינטראקטיבית עם PyVis

כיצד לקרוא את הרשת:

אלמנט	משמעות
גודל הצומת	PageRank – ערכים גדולים = חשובים יותר
צבע	קטגוריה: ארכיאולוגיה / היסטוריה / DH
קשתות כתומות	קישורים בין קטגוריות – מעניינים במיוחד!
קשתות אפורות	קישורים בתוך אותה קטגוריה

ניווט בויזואליזציה:

- **גלגל עכבר** = זום פנימה/חוצה
- **גרירה** = הזזת צמתים
- **לחיצה** = הדגשת הצומת והקשרים שלו
- **hover** = מידע מפורט על הצומת

```
In [ ... # == PyVis שקפים 16-17 == ויזואליזציה אינטראקטיבית ==
# =====
# שחזור מהיר מקבצים שמורים (דלג על שליפה מחדש)
# =====
import requests, json, pandas as pd, networkx as nx
from pyvis.network import Network
import time, os
```

```

from tqdm.auto import tqdm
from IPython.display import HTML, display
import matplotlib, matplotlib.pyplot as plt
from bidi.algorithm import get_display
matplotlib.rcParams['axes.unicode_minus'] = False

CATEGORIES = {
    'ארכאולוגיה של ארץ ישראל': {'color': '#e74c3c', 'bg_color': '#fadbd8', 'icon': '🏛️'},
    'תולדות עם ישראל': {'color': '#2980b9', 'bg_color': '#d6eaf8', 'icon': '📖'},
    'מדעי הרוח הדיגיטליים': {'color': '#27ae60', 'bg_color': '#d5f5e3', 'icon': '💻'}
}

# שמור CSV-טעינת נתוני ערכים מ
df = pd.read_csv('07_all_pages.csv', encoding='utf-8-sig')
print(f"📁 נטענו {len(df)} ערכים מ-07_all_pages.csv")

# שמור (הרבה יותר מהיר משליפה מחדש) GraphML-בניית גרף מ
G = nx.read_graphml('09_network.graphml')
# df-הוספת תכונות צבע+קטגוריה לצמתים מה
title_to_cat = dict(zip(df['title'], df['category']))
for node in G.nodes():
    cat = title_to_cat.get(node, '')
    G.nodes[node]['category'] = cat
    G.nodes[node]['color'] = CATEGORIES.get(cat, {}).get('color', '#95a5a6')
    G.nodes[node]['group'] = CATEGORIES.get(cat, {}).get('group', 1)
print(f"🕸 קשתות {G.number_of_edges()} צמתים {G.number_of_nodes()} גרף נטען")

# G על PageRank שמור + עדכון תכונות CSV-מ df_stats טעינת
df_stats = pd.read_csv('02_network_stats.csv', encoding='utf-8-sig')
for _, row in df_stats.iterrows():
    n = row['כותרת']
    if n in G.nodes():
        G.nodes[n]['in_degree'] = int(row['קישורים_נכנסים'])
        G.nodes[n]['out_degree'] = int(row['קישורים_יוצאים'])
        G.nodes[n]['degree'] = int(row['סך_קישורים'])
        G.nodes[n]['pagerank'] = float(row['PageRank'])
print(f"📊 02-נטענו מרכזיות מ-02_network_stats.csv")
print(f"✅ כל המשתנים מוכנים - ממשיכים לויזואליזציה")

```

נטענו 803 ערכים מ-vsc.segap_lla_70 📁

גרף נטען: 803 צמתים, 2171 קשתות 🕸

מדי מרכזיות נטענו מ-vsc.stats_krowten_20 📊

כל המשתנים מוכנים - ממשיכים לויזואליזציה ✅

```

gorPI :gninraWmdqT :12:yp.otua/mdqt/segakcap-etis/01.3nohtyp/bil/werbemoh/tpo/
r.stegdiwypi//:sptth eeS .stegdiwypi dna retypuj etadpu esaelP .dnuof ton sser
lmth.llatsni_resu/elbats/ne/oi.scodehtdae
mdqt_koobeton sa mdqt tropmi koobetonotua. morf

```

```

In [ ... # == PyVis שוקף 17 == ויזואליזציה אינטראקטיבית ==
# =====
# PyVis - ויזואליזציה אינטראקטיבית
# =====

```

```

import re, json, numpy as np

print(f"🌈...יוצר ויזואליזציה אינטראקטיבית")

```

```

# — 1. מהברירת מחדל למניעת חפיפה  $k \times 5$  – מיקומים מראש —————
n_nodes = len(G.nodes())
pos = nx.spring_layout(G, seed=42,
                       k=5.0 / np.sqrt(n_nodes),
                       iterations=150)

# — 2. בלבד (top-60 PageRank איזה צמתים יקבלו תווית) —————
label_set = set(df_stats.head(60)['כותרת'].tolist())

# — 3. (JS-ייצאן ל) מטא-נתונים לצמתים —————
pr_rank = {row['כותרת']: i + 1 for i, row in df_stats.iterrows()}
node_meta = {}
for node in G.nodes():
    nd = G.nodes[node]
    node_meta[node] = {
        'cat': nd.get('category', ''),
        'color': nd.get('color', '#95a5a6'),
        'in': nd.get('in_degree', 0),
        'out': nd.get('out_degree', 0),
        'pr': round(nd.get('pagerank', 0), 6),
        'rank': pr_rank.get(node, 0),
    }
node_meta_json = json.dumps(node_meta, ensure_ascii=False)

# — 4. PyVis בניית רשת —————
net = Network(
    height='700px',
    width='100%',
    bgcolor='#f8f9fa',
    font_color='#2c3e50',
    directed=True
)

max_pr = max(G.nodes[n].get('pagerank', 0) for n in G.nodes()) or 1

for node in G.nodes():
    nd = G.nodes[node]
    pr = nd.get('pagerank', 0)
    color = nd.get('color', '#95a5a6')
    size = 7 + (pr / max_pr) * 36
    # קצורות ל-18 תווים, top-60 תוויות: רק
    lbl = node[:18] if node in label_set else ''
    x, y = pos[node]
    # ⚠ מותאם JS מנוהל ע"י פאנל (tooltip title=
    # ⚠ vis.js-מחליף צבעים ב) group=
    net.add_node(
        node,
        label=lbl,
        color={
            'background': color,
            'border': color,
            'highlight': {'background': '#f1c40f', 'border': '#e67e22'},
            'hover': {'background': '#f9e79f', 'border': color},
        },
        size=size,
    )

```

```

        x=float(x) * 1600,
        y=float(y) * 1600,
        physics=False,
    )

# — 5. קשתות —————
for src, tgt in G.edges():
    cross = G.nodes[src].get('category', '') != G.nodes[tgt].get('category', '')
    net.add_edge(
        src, tgt,
        color='#e67e22' if cross else '#c8cfd8',
        width=2.2 if cross else 0.6,
    )

# — 6. רציוני HTML כתיבת —————
net.write_html('03_knowledge_network.html')

with open('03_knowledge_network.html', 'r', encoding='utf-8') as f:
    html = f.read()

# — 7. options (set_options נדחה ע"י pyvis) —————
correct_options = {
    "physics": {"enabled": False, "stabilization": {"enabled": False}},
    "nodes": {"font": {"size": 11, "face": "Arial", "color": "#2c3e50"},
              "borderWidth": 1, "borderWidthSelected": 3,
              "shadow": {"enabled": True, "size": 3, "x": 2, "y": 2}},
    "edges": {"smooth": {"type": "continuous"}, "color": {"inherit": False},
              "arrows": {"to": {"enabled": True, "scaleFactor": 0.3}},
              "shadow": False},
    "interaction": {"hover": True, "tooltipDelay": 9999, # ← בנה tooltip מבטל
                    "navigationButtons": False, "keyboard": True,
                    "zoomView": True, "dragView": True},
    "configure": {"enabled": False},
}
html = re.sub(r'var options = \{.*?\};',
              f'var options = {json.dumps(correct_options)};',
              html, flags=re.DOTALL)

# — 8. הסרת loadingBar —————
html = re.sub(r'<div id="loadingBar".*?</div>\s*</div>', '', html, flags=re.DOTALL)

# — 9. JS: hover-focus + info-panel + click —————
hover_js = f"""
var nodeMetadata = {node_meta_json};
var totalNodes = {n_nodes};

// שמירת צבעים מקוריים לשחזור
var _origNC = {}, _origEC = {};
nodes.forEach(function(n) {{
    _origNC[n.id] = n.color ? JSON.parse(JSON.stringify(n.color)) : null;
}});
edges.forEach(function(e) {{
    _origEC[e.id] = e.color ? JSON.parse(JSON.stringify(e.color)) : null;
}});

var DIM_NODE = {{ background:'#e8e8e8', border:'#d0d0d0',

```

```

        highlight:{{background:'#e8e8e8',border:'#d0d0d0'}},
        hover:{{background:'#e8e8e8',border:'#d0d0d0'}} }};
var DIM_EDGE = {{ color:'#ebebeb', highlight:'#ebebeb', hover:'#ebebeb', inheri
var catColors = {{ 'ארץ ישראל של ארץ ישראל': '#e74c3c',
                    'ב9תולדות עם ישראל': '#2980b9',
                    'ae60הרוח הדיגיטליים': '#27ae60'}};

function showPanel(hId) {{
    var nd = nodeMetadata[hId];
    if (!nd) return;
    var cc = catColors[nd.cat] || '#95a5a6';
    var conn = network.getConnectedNodes(hId).length;
    document.getElementById('nip-title').textContent = hId;
    document.getElementById('nip-cat').innerHTML =
        '<span style="display:inline-block;width:10px;height:10px;border-radius:50%
        'background:' + cc + ';margin-left:6px;vertical-align:middle"></span>' + nd
    document.getElementById('nip-stats').innerHTML =
        '<tr><td class="nip-lbl">👤 קישורים נכנסים</td>' +
        '<td class="nip-val">' + nd.in + '</td></tr>' +
        '<tr><td class="nip-lbl">👤 קישורים יוצאים</td>' +
        '<td class="nip-val">' + nd.out + '</td></tr>' +
        '<tr><td class="nip-lbl">📊 PageRank</td>' +
        '<td class="nip-val" style="color:#e67e22;font-weight:bold">' +
        nd.pr.toFixed(5) + '</td></tr>' +
        '<tr><td class="nip-lbl">🏆 דירוג</td>' +
        '<td class="nip-val">#<' + nd.rank + ' / ' + totalNodes + '</td></tr>' +
        '<tr><td class="nip-lbl">🌐 שכנים ישירים</td>' +
        '<td class="nip-val">' + conn + '</td></tr>';
    document.getElementById('nip-footer').textContent = '← לפתיחה בוויקיפדיה
    document.getElementById('node-info-panel').style.display = 'block';
}}

function hidePanel() {{
    document.getElementById('node-info-panel').style.display = 'none';
}}

network.on('hoverNode', function(params) {{
    var hId      = params.node;
    var neighbors = new Set(network.getConnectedNodes(hId));
    neighbors.add(hId);
    var connEdges = new Set(network.getConnectedEdges(hId));

    // עמסום צמתים לא-מחוברים
    var nu = [];
    nodes.forEach(function(n) {{
        if (!neighbors.has(n.id))
            nu.push({{ id: n.id, color: DIM_NODE, font: {{ color: '#cccccc' }} }});
    }});
    nodes.update(nu);

    // עמסום קשתות לא-מחוברות
    var eu = [];
    edges.forEach(function(e) {{
        if (!connEdges.has(e.id)) eu.push({{ id: e.id, color: DIM_EDGE }});
    }});
    edges.update(eu);

```

```

        document.getElementById('mynetwork').style.cursor = 'pointer';
        showPanel(hId);
    }));

network.on('blurNode', function() {{
    // שחזור צמתים
    var nu = [];
    nodes.forEach(function(n) {{
        var u = {{ id: n.id, font: {{ color: '#2c3e50' }} }};
        if (_origNC[n.id]) u.color = _origNC[n.id];
        nu.push(u);
    }});
    nodes.update(nu);

    // שחזור קשתות
    var eu = [];
    edges.forEach(function(e) {{
        var u = {{ id: e.id }};
        if (_origEC[e.id]) u.color = _origEC[e.id];
        eu.push(u);
    }});
    edges.update(eu);

    document.getElementById('mynetwork').style.cursor = 'default';
    hidePanel();
}});

// לחיצה – פתיחת הערך בוויקיפדיה
network.on('click', function(params) {{
    if (params.nodes.length > 0) {{
        var title = params.nodes[0];
        window.open('https://he.wikipedia.org/wiki/' +
            encodeURIComponent(title.replace(/ /g, '_')), '_blank');
    }}
}});
""""

html = html.replace(
    'network = new vis.Network(container, data, options);',
    'network = new vis.Network(container, data, options);\n' + hover_js
)

# — 10. אחרית אתחול + fit כיבוי פיזיקה
stop_js = ""
<script>
document.addEventListener('DOMContentLoaded', function () {
    var t = setInterval(function () {
        if (typeof network !== 'undefined') {
            clearInterval(t);
            network.setOptions({ physics: { enabled: false } });
            network.stopSimulation();
            setTimeout(function () {
                network.fit({ animation: { duration: 700, easingFunction: 'easeInOutQua
                    }, 200);
            }

```

```

    }, 50);
});
</script>
"""
html = html.replace('</body>', stop_js + '\n</body>')

# — 11. פאנל מיידע + מקרא —————
info_panel = """
<style>
  #node-info-panel { pointer-events:none; }
  .nip-lbl { color:#888; padding:3px 12px 3px 0; white-space:nowrap; }
  .nip-val { font-weight:600; text-align:left; padding:3px 0; }
</style>
<div id="node-info-panel" style="
  display:none; position:absolute; top:12px; left:14px;
  background:rgba(255,255,255,0.97);
  border:1px solid #dde; border-radius:10px;
  padding:14px 18px; font-family:Arial,sans-serif;
  font-size:13px; direction:rtl; min-width:255px;
  box-shadow:0 4px 16px rgba(0,0,0,0.14); z-index:998;">
  <div id="nip-title" style="font-weight:bold;font-size:15px;margin-bottom:8px;
    color:#2c3e50;padding-bottom:7px;border-bottom:2px solid #eee;word-break
  <div id="nip-cat" style="margin-bottom:10px;font-size:12px;color:#555;"></div>
  <table id="nip-stats" style="width:100%;border-collapse:collapse;font-size:12
  <div id="nip-footer" style="font-size:10px;color:#aaa;margin-top:10px;
    padding-top:6px;border-top:1px solid #eee;"></div>
</div>
"""

legend = """
<div id="wiki-legend" style="
  position:absolute; top:12px; right:14px;
  background:rgba(255,255,255,0.95);
  border:1px solid #dde; border-radius:8px;
  padding:12px 16px; font-family:Arial,sans-serif;
  font-size:12px; direction:rtl; min-width:210px;
  box-shadow:0 2px 8px rgba(0,0,0,0.12); z-index:999;">
  <div style="font-weight:bold;margin-bottom:8px;font-size:13px;">מקרא</div>
  <div style="margin:5px 0">
    <span style="display:inline-block;width:13px;height:13px;border-radius:50%;
  </div>
  <div style="margin:5px 0">
    <span style="display:inline-block;width:13px;height:13px;border-radius:50%;
  </div>
  <div style="margin:5px 0">
    <span style="display:inline-block;width:13px;height:13px;border-radius:50%;
  </div>
  <hr style="border:none;border-top:1px solid #eee;margin:9px 0">
  <div style="color:#777;font-size:11px;line-height:1.7">
    <b>גודל הצומת</b> = PageRank<br>
    <b style="color:#e67e22">ביון-קטגוריאלי</b> = קשת כתומה<br>
    <b style="color:#b0b8c4">פנים-קטגוריאלי</b> = קשת אפורה<br>
    <span style="color:#aaa;font-size:10px"> • גלגל עכבר לזום • לחצו לוויקיפדיה
  </div>
</div>
"""

```

```

html = html.replace('<div id="mynetwork"', info_panel + legend + '\n<div id="my
with open('03_knowledge_network.html', 'w', encoding='utf-8') as f:
    f.write(html)

print("✅ 03 הרשת נשמרה: knowledge_network.html")
print()
print("📄 שיפורים שהוחלו:")
print(f"✓ מהברירת מחדל, סקאל  $5 \times 1600 \times k$  - פריסה מרווחת")
print(f"✓ (סה"כ {n_nodes}) צמתים מרכזיים  $\{\min(60, n\_nodes)\}$ -תוויות רק ל")
print("✓ hover: עמעום שכנים לא-מחוברים + שחזור בעת הפסקת")
print("✓ פאנל מידע: כותרת, קטגוריה, מדדים, דירוג, מספר שכנים")
print("✓ לחיצה על צומת → פתיחת הערך בוויקיפדיה")
print("✓ מוחלף בפאנל מותאם vis.js מובנה של tooltip")
print()

display(HTML(html))

```

יוצר ויזואליזציה אינטראקטיבית...
 הרשת נשמרה: lmth.krowten_egdelwonk_30 ✅

שיפורים שהוחלו: 📄

- ✓ פריסה מרווחת - $5 \times k$ מהברירת מחדל, סקאל $1600 \times 5 \times k$
- ✓ תוויות רק ל-06 צמתים מרכזיים (803 סה"כ)
- ✓ revoh: עמעום שכנים לא-מחוברים + שחזור בעת הפסקת revoh
- ✓ פאנל מידע: כותרת, קטגוריה, מדדים, דירוג, מספר שכנים
- ✓ לחיצה על צומת → פתיחת הערך בוויקיפדיה
- ✓ pitloot מובנה של sj.siv מוחלף בפאנל מותאם

מקרא

ארכאולוגיה של ארץ ישראל ●

תולדות עם ישראל ●

מדעי הרוח הדיגיטליים ●

גודל הצומת = PageRank

קשת כתומה = בין-קטגוריאלי

קשת אפורה = פנים-קטגוריאלי

גרור · גלגל עכבר לזום · לחצו לזוויקיפדיה

חלק ה: ניתוח פערים – מה חסר בוויקיפדיה?

שיטות לזיהוי פערים:

1. קישורים אדומים (Red Links): ערכים שמוזכרים בטקסט אך לא קיימים
2. רשימת מומחים: נושאים חשובים לתחום שלא מתועדים
3. ניתוח הרשת: אזורים דלילים, ערכים מבודדים

הפרויקט שלך: 💡

כל ערך חסר = פרויקט פוטנציאלי!

ניתן לכתוב ערך חדש, לשפר ערך קיים, או לתרגם מוויקיפדיה אחרת.

"The sum of all human knowledge" – הסיסמה של ויקיפדיה

אבל מה לא נמצא שם עדיין?

In [...

```
# == קישורים אדומים == 18 שקף ==
# =====
# ניתוח פערים – קישורים אדומים
# =====

print("🔴 (ערכים חסרים) מחפש קישורים אדומים...")
print("...בודק את 15 הערכים המרכזיים ביותר")
print()

top_pages = df_stats.head(15)['כותרת'].tolist()
red_link_data = []

for title in top_pages:
    missing = wiki.get_red_links(title)
    cat = G.nodes[title].get('category', '') if title in G.nodes() else ''
    icon = CATEGORIES.get(cat, {}).get('icon', '•')

    print(f" {icon} {title[:38]:<38} → {len(missing)} קישורים אדומים")
    for m in missing[:5]:
        red_link_data.append({
            'ערך_חסר': m,
            'מוזכר_ב': title,
            'קטגוריה': cat
        })

if red_link_data:
    df_red = pd.DataFrame(red_link_data)
    top_missing = df_red.groupby('ערך_חסר').size().sort_values(ascending=False)

    print(f"\n📊 ערכים חסרים עם הכי הרבה אזכורים:")
    print("-" * 55)
    for topic, count in top_missing.head(15).items():
        print(f"❌ {topic[:48]:<48} ({count}x)")

    df_red.to_csv('04_red_links.csv', index=False, encoding='utf-8-sig')
    print(f"\n📁 04 :נשמר_red_links.csv")
else:
    print(f"\n⚠️ לא נמצאו קישורים אדומים בדגימה זו")
```

● מחפש קישורים אדומים (ערכים חסרים)...
בודק את 51 הערכים המרכזיים ביותר...

פרויקט גוטנברג	→ 21 קישורים אדומים
ארכיון האינטרנט	→ 3 קישורים אדומים
פרויקט בן-יהודה	→ 31 קישורים אדומים
דעת (אתר אינטרנט)	→ 2 קישורים אדומים
ויקיטקסט	→ 11 קישורים אדומים
אנטישמיות	→ 71 קישורים אדומים
היברובוקס	→ 1 קישורים אדומים
ספרייה דיגיטלית	→ 2 קישורים אדומים
גוגל ספרים	→ 41 קישורים אדומים
פרויקט השו"ת	→ 5 קישורים אדומים
אוצר הספרים היהודי השיתופי	→ 2 קישורים אדומים
ספריית אסיף	→ 9 קישורים אדומים
המילון ההיסטורי ללשון העברית	→ 1 קישורים אדומים
אוצר החכמה	→ 2 קישורים אדומים
על התורה	→ 1 קישורים אדומים

🇮🇱 ערכים חסרים עם הכי הרבה אזכורים:

אוצרות התורה	(x9)	✖
kooBnoitciF	(x4)	✖
redaeR tfosorciM	(x4)	✖
tekcopiboM	(x3)	✖
kooN	(x2)	✖
מפתח (ספרנות)	(x1)	✖
יריד הספרים בפרנקפורט	(x1)	✖
מאגר דיגיטלי	(x1)	✖
מאמינים במשטרה	(x1)	✖
מייקל הארט	(x1)	✖
מכון משפט לעם	(x1)	✖
עמותה למחשוב ספרות עברית	(x1)	✖
עמוד (נייר)	(x1)	✖
יהודה איזנברג	(x1)	✖
קמפוס - המיזם הלאומי ללמידה דיגיטלית	(x1)	✖

📁 נשמר: vsc.sknil_der_40

```
In [ ... # == שקף 19 == הצעות לפרויקטי סיום ==  
# =====  
# הצעות לפרויקטים - בדיקת נושאים חסרים  
# =====  
  
SUGGESTED_PROJECTS = {  
    'ארכאולוגיה של ארץ ישראל': [  
        'ארכאולוגיה תת-ימית בישראל',  
        'ארכאולוגיה-בוטניקה בחפירות ישראל',  
        'ניתוח איזוטופים בארכאולוגיה',  
        'ארכאולוגיה דיגיטלית בישראל',  
        'ארכאולוגיה של הנוף בארץ ישראל',  
        'ארכאולוגיה ביזנטית בנגב',  
        'ארכאולוגיה של ימי הביניים בארץ ישראל',  
        'ארכאולוגיה ישראלית LiDAR',  
        'ניהול מורשת תרבותית בישראל',  
        'ארכאולוגיה של מסחר עתיק בארץ ישראל',  
    ]  
}
```

```

],
'תולדות עם ישראל': [
    'היסטוריוגרפיה יהודית מודרנית',
    'יהדות אתיופיה – היסטוריה',
    'יהדות תימן – ההגירה לישראל',
    'יהודי אמריקה הלטינית – היסטוריה',
    'תולדות הדפוס העברי',
    'נשים בהיסטוריה היהודית',
    'יהדות איראן בעת החדשה',
    'יהודי מרוקו בישראל',
    'ספרות יהודית-ערבית',
    'יהדות הודו – קהילות ופזורה',
],
'מדעי הרוח הדיגיטליים': [
    'עיבוד שפה טבעית לעברית',
    'דיגיטציה של כתבי יד עבריים',
    'GIS בארכיאולוגיה',
    'לחוקרי מדעי הרוח Wikidata',
    'במחקר ישראלי Open Access',
    'ניתוח רשת בהיסטוריה',
    'בינה מלאכותית בפרשנות טקסטים',
    'מפות דיגיטליות לחקר ארץ ישראל',
    'הגניזה הקהירית – פרויקטים דיגיטליים',
    'לכתב יד עברי OCR',
],
]
}

print("🔍...בודק אילו נושאים מוצעים חסרים בוויקיפדיה")
print("=" * 65)

project_list = []
for category, topics in SUGGESTED_PROJECTS.items():
    print(f"\n{CATEGORIES[category]['icon']} {category}:")

    existence = wiki.check_exists(topics)

    for topic in topics:
        exists = existence.get(topic, False)
        status = "✅ קיים" if exists else "❌ חסר →"
        note = "הזדמנות לפרויקט ←" if not exists else ""
        print(f"    {status} {topic:<48}{note}")

        project_list.append({
            'נושא': topic,
            'קטגוריה': category,
            'קיים_בוויקיפדיה': exists,
            'הצעה_לפרויקט': not exists
        })

    time.sleep(0.5)

df_proj = pd.DataFrame(project_list)
missing_n = (~df_proj['קיים_בוויקיפדיה']).sum()

print(f"\n{'='*65}")
print(f"📊 נושאים חסרים בוויקיפדיה {len(df_proj)} מתוך {missing_n} סיכום")

```

```
print(f" → {missing_n} סיום !הזדמנויות לפרויקטי סיום")

df_proj.to_csv('05_project_suggestions.csv', index=False, encoding='utf-8-sig')
print("\n📁 05 : נשמר project_suggestions.csv")
```

🔗 בודק אילו נושאים מוצעים חסרים בוויקיפדיה...

📁 ארכאולוגיה של ארץ ישראל:

חסר →	ארכאולוגיה תת-ימית בישראל	✗
חסר →	ארכאולוגיה בוסטניקה בחפירות ישראל	✗
חסר →	ניתוח איזוטופים בארכאולוגיה	✗
חסר →	ארכאולוגיה דיגיטלית בישראל	✗
חסר →	ארכאולוגיה של הנוף בארץ ישראל	✗
חסר →	ארכאולוגיה ביזנטית בנגב	✗
חסר →	ארכאולוגיה של ימי הביניים בארץ ישראל	✗
חסר →	RADiL בארכאולוגיה ישראלית	✗
חסר →	ניהול מורשת תרבותית בישראל	✗
חסר →	ארכאולוגיה של מסחר עתיק בארץ ישראל	✗

📁 תולדות עם ישראל:

חסר →	היסטוריוגרפיה יהודית מודרנית	✗
חסר →	יהדות אתיופיה – היסטוריה	✗
חסר →	יהדות תימן – ההגירה לישראל	✗
חסר →	יהודי אמריקה הלטינית – היסטוריה	✗
קיים ✓	תולדות הדפוס העברי	✓
חסר →	נשים בהיסטוריה היהודית	✗
חסר →	יהדות איראן בעת החדשה	✗
חסר →	יהודי מרוקו בישראל	✗
חסר →	ספרות יהודית-ערבית	✗
חסר →	יהדות הודו – קהילות ופזורה	✗

📁 מדעי הרוח הדיגיטליים:

חסר →	עיבוד שפה טבעית לעברית	✗
חסר →	דיגיטציה של כתבי יד עבריים	✗
חסר →	SIG בארכיאולוגיה	✗
חסר →	atadikiW לחוקרי מדעי הרוח	✗
חסר →	ssecca nep0 במחקר ישראלי	✗
חסר →	ניתוח רשת בהיסטוריה	✗
חסר →	בינה מלאכותית בפרשנות טקסטים	✗
חסר →	מפות דיגיטליות לחקר ארץ ישראל	✗
חסר →	הגניזה הקהירית – פרויקטים דיגיטליים	✗
חסר →	RC0 לכתב יד עברי	✗

📁 סיכום: 92 מתוך 03 נושאים חסרים בוויקיפדיה
→ 92 הזדמנויות לפרויקטי סיום!

📁 נשמר: vsc.snoitseggus_tcejorp_50

```
In [ ... # == שמירת כל הנתונים == שקף 20 ==
# =====
# שמירת כל הנתונים
# =====

print("\n📁 שומר נתונים ועצמי גרף")
```

```
df.to_csv('07_all_pages.csv', index=False, encoding='utf-8-sig')
df_stats.to_csv('02_network_stats.csv', index=False, encoding='utf-8-sig')

# קבצי גרף לניתוח בכלים חיצוניים (Gephi, Cytoscape)
nx.write_gexf(G, '08_network.gexf')
nx.write_graphml(G, '09_network.graphml')

print("\n📁 קבצים שנוצרו:")
for fname in sorted(os.listdir('.')):
    if any(fname.endswith(ext) for ext in ['.csv', '.html', '.gexf', '.graphml']):
        size = os.path.getsize(fname)
        print(f"📄 {fname:<42} {size:>10,} bytes")

print("\n🎉 מחברת 1 הושלמה!")
print()
print("📖 המשך:")
print("1 📄 05 - project_suggestions.csv - בחרו נושא")
print("2 📄 03 - knowledge_network.html - חקרו את הרשת")
print("3 📄 ! עברו למחברת 2: קריאה מרחוק")
```

📁 שומר נתונים ועצמי גרף...

setyb 789,95
setyb 634,52
setyb 121,197
setyb 622,4
setyb 849,2
setyb 797,22
setyb 580,343
setyb 736,072

📁 קבצים שנוצרו:

- gnp.noitubirtsid_yrogetac_10 📄
- vsc.stats_krowten_20 📄
- lmth.krowten_egdelwonk_30 📄
- vsc.sknil_der_40 📄
- vsc.snoitseggus_tcejorp_50 📄
- vsc.segap_lla_70 📄
- fxeg.krowten_80 📄
- lmhparg.krowten_90 📄

🎉 מחברת 1 הושלמה!

📖 המשך:

- 1 📄 עיינו ב-vsc.snoitseggus_tcejorp_50 - בחרו נושא
- 2 📄 פתחו את lmth.krowten_egdelwonk_30 - חקרו את הרשת
- 3 📄 עברו למחברת 2: קריאה מרחוק! →

📝 תרגילים

🌟 תרגיל 1 - בסיסי

בחרו קטגוריה שמעניינת אתכם ובצעו ניתוח דומה:

```
# שנו לקטגוריה שבחרתם:
pages = wiki.get_category_members("ארכיאולוגיה של ים המלח")
```

🌟🌟 תרגיל 2 - בינוני

הוסיפו לויזואליזציה **מקרא (Legend)** המסביר את הצבעים, הצורות, וגודל הצמתים.
רמז: PyVis תומך ב-HTML מותאם אישית בתיאור הרשת.

תרגיל 3 – מתקדם ⭐⭐⭐

השוו את הרשת העברית לרשת האנגלית לאותן קטגוריות.
שנו את `BASE_URL` ל- `https://en.wikipedia.org/w/api.php` וחפשו:

- אילו ערכים קיימים באנגלית אבל לא בעברית?

משאבים 🔗

- [MediaWiki API Documentation](#)
- [NetworkX Documentation](#)
- [PyVis Documentation](#)
- [\(Barabási, חינום\) Network Science](#)
- [Programming Historian](#) – שיעורי DH

מטלת בית – הגשה 🏠

מה להגיש:

1. **צילום מסך** של הרשת האינטראקטיבית שיצרתם (קובץ `.png` / `.jpg`)
2. **קובץ ה-CSV** של הניתוח הסטטיסטי (`02_network_stats.csv`)
3. **תשובה קצרה** (3–5 משפטים) לאחת מהשאלות הבאות:
 - מהו הערך המרכזי ביותר ברשת, ולמה לדעתכם?
 - מהו פרויקט ויקיפדיה שהייתם רוצים לכתוב לאחר ראיית הפערים?

כיצד להגיש:

דרך הטופס בקישור בעמוד המטלה באתר הקורס.

צריכים עזרה?

שאלו את [Google Gemini](#) באופן הבא:

"אני עובד/ת עם *NetworkX* בפיתוח ומנסה [מה שאתם מנסים]. קיבלתי שגיאה: [העתיקו כאן].
מה הבעיה ואיך לתקן?"

רשימת בדיקה לפני הגשה ✅

- ☐ הרצתי את כל התאים מלמעלה למטה
- ☐ יש לי גרף ויזואלי של הרשת
- ☐ שמרתי את קובץ ה-CSV
- ☐ כתבתי תשובה לאחת השאלות